Explainable Context-Aware Sprint Planning With Real-World Constraints

SoftwareDesignDocument

Group Name: Team ApexS

Madhan karthikeyan	**23BCE0829**	**madhan.karthikeyan2023@vitstudent.ac.in**
Midhul Kiruthik M (Team Leader)	**23BCE0783**	**midhulkiruthik.m2023@vitstudent.ac.in**
Manav Kaizen	**23BCE2251**	**manav.kaizen2023@vitstudent.ac.in**
Sharvesh S	**23BCE2141**	**sharvesh.s2023@vitstudent.ac.in**

Instructor:	**SWARNALATHA P**
Course:	**Software Engineering Lab**
Lab Section:	**L23+L24**
Date:	**18/02/2026**

TABLEOFCONTENTS

1. INTRODUCTION

Purpose
Scope
Overview
ReferenceMaterial
DefinitionsandAcronyms

2. SYSTEMOVERVIEW

3. SYSTEMARCHITECTURE

ArchitecturalDesign
DecompositionDescription
DesignRationale

4. DATADESIGN

DataDescription
DataDictionary

5. COMPONENTDESIGN

6. HUMANINTERFACEDESIGN

OverviewofUserInterface
ScreenImages
ScreenObjectsandActions

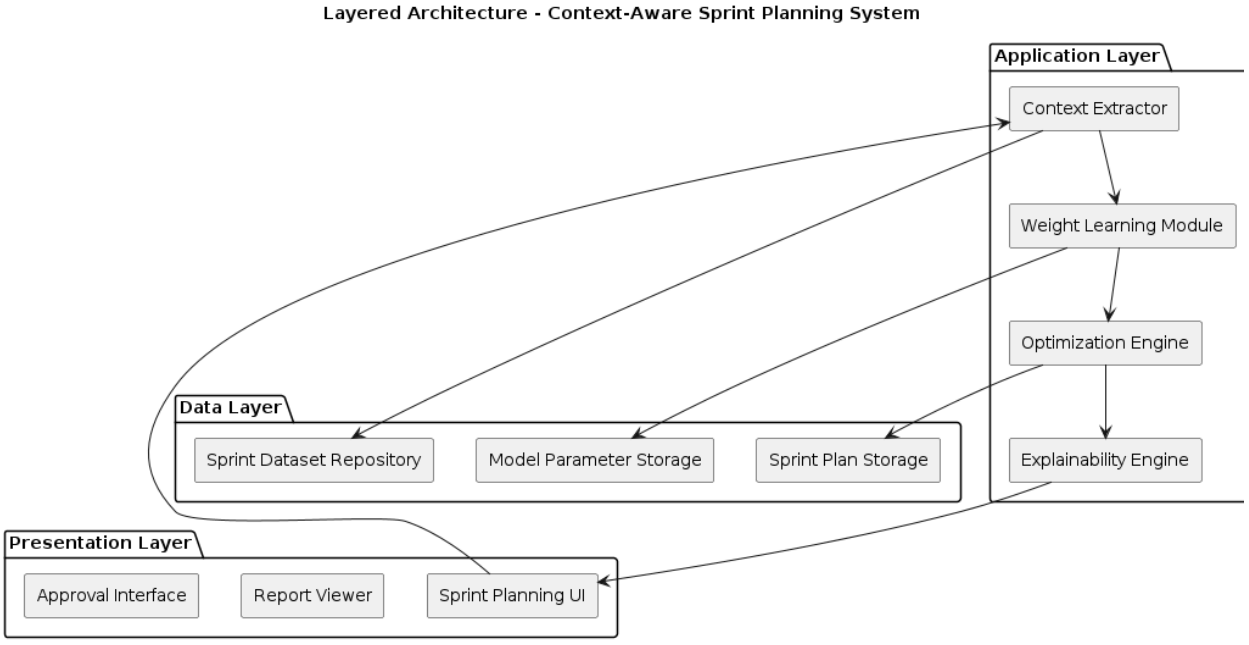
7. REQUIREMENTSMATRIX

8. APPENDICES

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the internal architecture and detailed design of the **Context-Aware Automated Sprint Planning System**. It translates the functional and non-functional requirements from the SRS into architecture, components, data models, algorithms, and interfaces required for implementation. This document is the primary technical reference for developers, maintainers, and evaluators.



Layered system architecture

1.2 Scope

The system is a decision-support tool for Scrum teams to generate optimized sprint backlogs using historical sprint data, contextual information, and multi-objective optimization. It includes risk-awareness, skill-awareness, and explainability components to improve planning quality and trust.

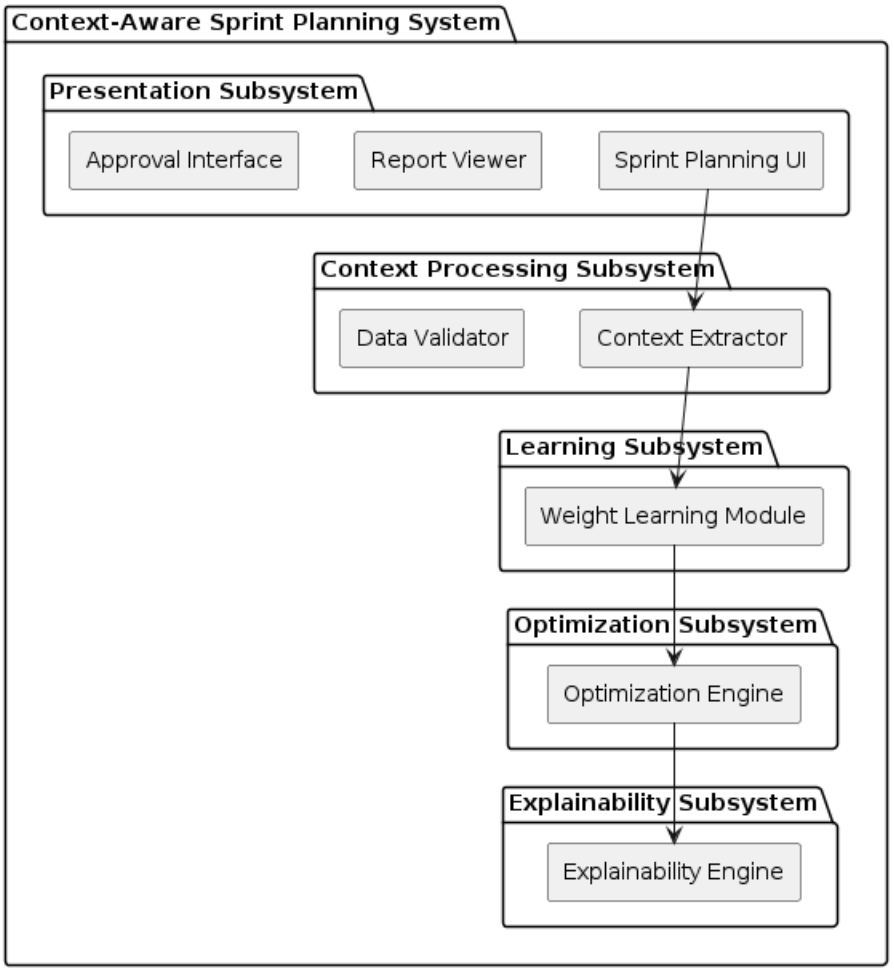
This SDD specifies internal structure and algorithms necessary to satisfy the SRS. It covers only the sprint-planning subsystem (not a full PM tool).

1.3 Overview

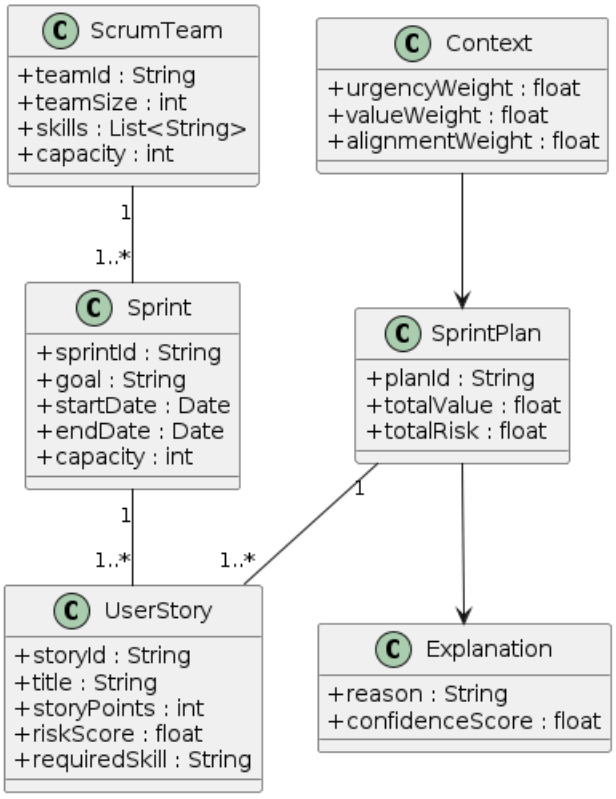
This document contains:

- System overview
- Architectural design and decomposition
- Data design and dictionary
- Detailed component designs (algorithms and interfaces)
- Human interface design (UI mockups)
- Requirements traceability matrix
- Appendices for data, algorithms, and UML artifacts

Module Decomposition - Context-Aware Sprint Planning System



UML Class Diagram - Context-Aware Sprint Planning System



1.4 Reference Material

- Software Requirements Specification (SRS) — Context-Aware Sprint Planning System
- IEEE Std 1016 — Software Design Description
- `SDD_Template` (provided)
- ASE 2024: *Context-Aware Automated Sprint Plan Generation* (base paper)

1.5 Definitions and Acronyms

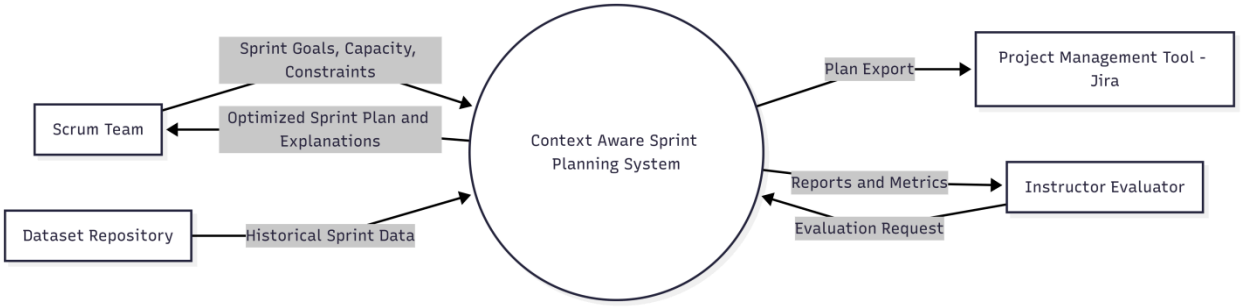
Term	Definition
SDS	Software Design Specification
SRS	Software Requirements Specification
ILP	Integer Linear Programming
UI	User Interface
Sprint Plan	Selected set of user stories for a sprint
Context	Project-specific environmental variables used for planning
Risk Score	Numerical estimate of delay or failure probability
Skill Match	Compatibility measure between story requirements and team skills

2. SYSTEMOVERVIEW

The system provides a web-based environment where Scrum teams can:

- Import historical sprint datasets
- Configure sprint context and constraints
- Generate optimized sprint backlogs
- Inspect explainable justifications for selections
- Export approved sprint plans to project tools (e.g., Jira)

The system follows a layered modular architecture integrating data ingestion, learning, optimization, and explainability modules.



Level-0 Context Diagram showing external interactions of the Context-Aware Sprint Planning System.

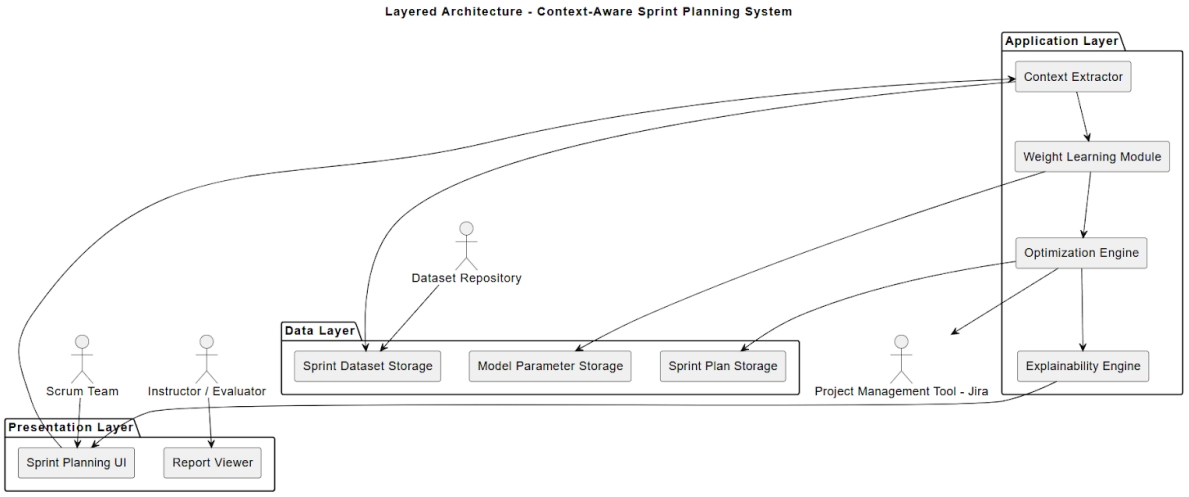
3. SystemArchitecture

3.1 Architectural Design

Architecture style: Layered modular architecture.
Layers and primary responsibilities:

- **Presentation Layer** — Sprint Planning UI, Report Viewer, Approval UI.
  Placeholder: `fig_ui_plan.png`, `fig_ui_upload.png` (Section 6).
- **Application Layer** — Context Extractor, Weight Learning Module, Optimization Engine, Explainability Engine.
- **Data Layer** — Sprint Dataset Repository, Sprint Plan Storage, Model Parameter Storage.

High-level collaboration flow:
UI → Context Extractor → Weight Learning → Optimization → Explainability → UI / Export to Jira.

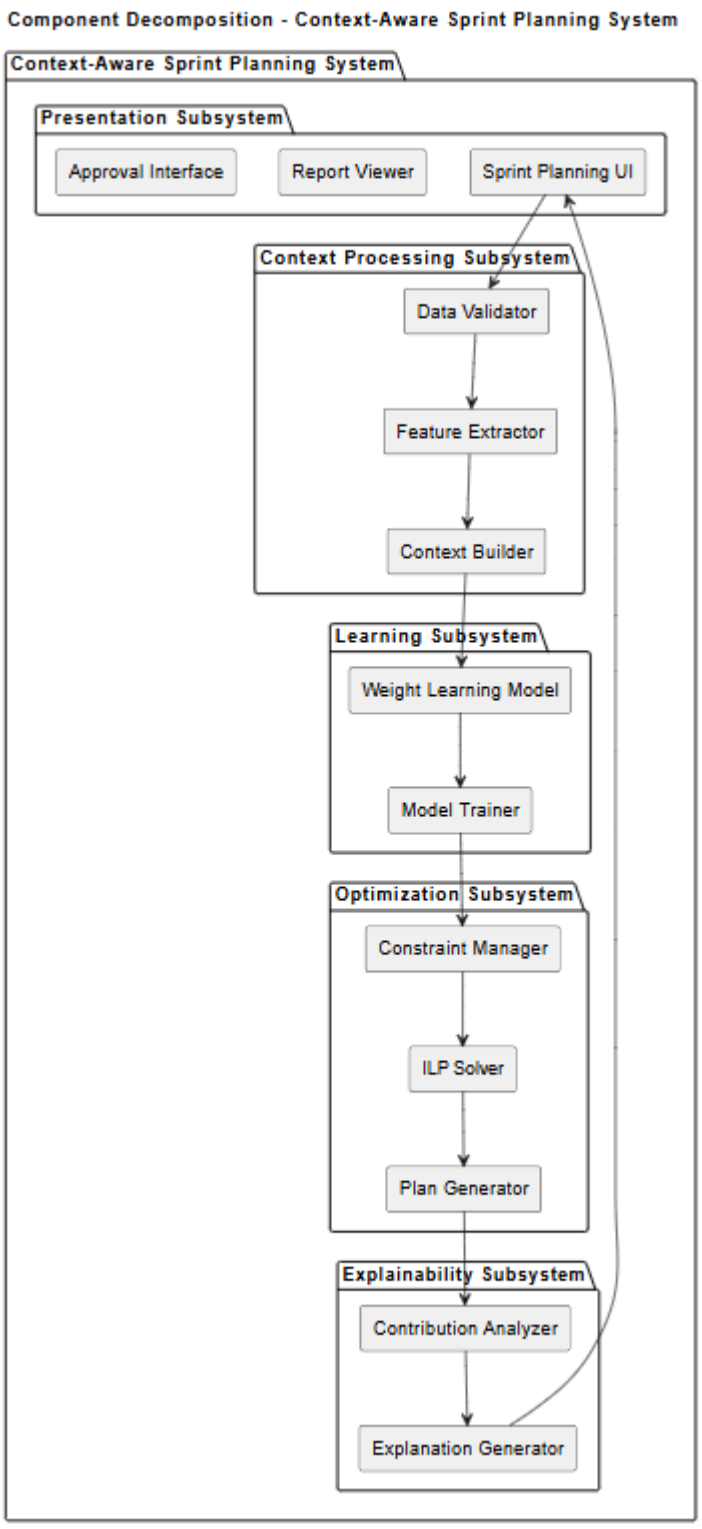


Layered Architecture of the Context-Aware Sprint Planning System illustrating presentation, application, and data layers along with major external interfaces.

3.2 Decomposition Description (Object-Oriented)

Subsystems and responsibilities:

1. **Presentation Subsystem** — Handles user workflows: upload data, configure sprint, view plan and explanations, approve/export.
   Placeholder: `fig_ui_plan.png`.
2. **Context Processing Subsystem** — Validation, normalization, feature extraction from historical sprint data.
3. **Learning Subsystem** — Trains or infers team-specific prioritization weights (urgency, value, alignment).
4. **Optimization Subsystem** — Solves constrained selection (ILP / heuristic) to maximize weighted value under capacity/dependency constraints and considers risk/skill.
5. **Explainability Subsystem** — Produces per-story justifications and confidence scores.



Component Decomposition Diagram illustrating internal subsystems and their interactions within the Context-Aware Sprint Planning System.

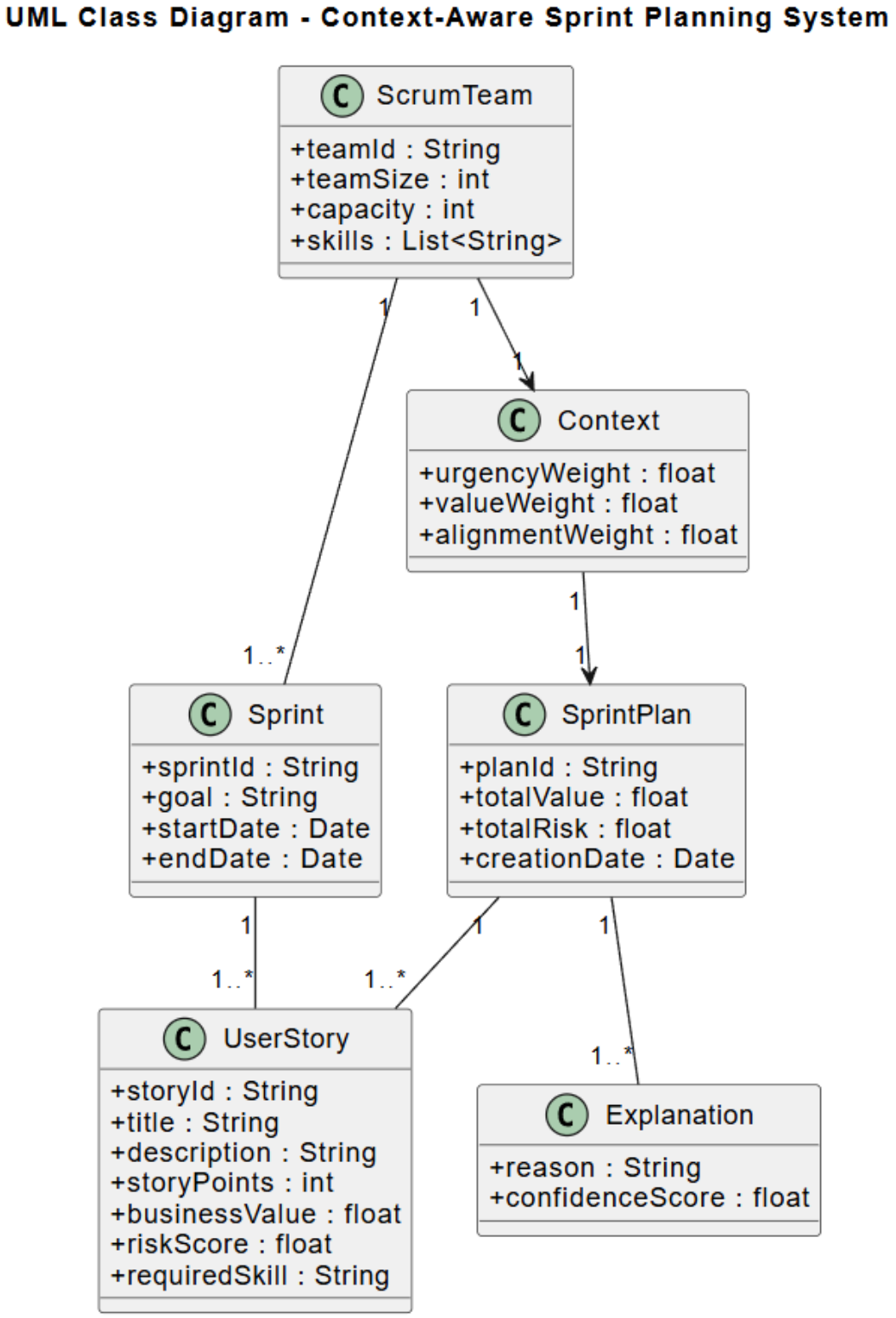
3.3 Design Rationale

- **Why layered?** Promotes modularity, testability, and future extension (new criteria, multi-team support).
- **Why OO?** Natural mapping of domain entities (Sprint, UserStory, Team) and reuse of components.
- **Alternatives considered:** Monolithic app (rejected for maintainability), microservices (rejected due to unnecessary operational complexity for an academic prototype).

4. DataDesign

4.1 Data Description

Major entities: Sprint, UserStory, ScrumTeam, Context, SprintPlan, Explanation.
Typical data flow:
Dataset → ContextExtractor → LearningModel → OptimizationEngine → SprintPlan → ExplainabilityEngine → UI / Export



UML Class Diagram representing the core domain entities and their relationships within the Context-Aware Sprint Planning System.

4.2 Data Dictionary

ScrumTeam

- `teamId : String` — unique team identifier
- `teamSize : Integer`
- `skills : List<String>` — skill tags available in team
- `capacity : Integer` — sprint effort capacity (story points)

Sprint

- `sprintId : String`
- `goal : String`
- `startDate : Date`
- `endDate : Date`

UserStory

- `storyId : String`
- `title : String`
- `storyPoints : Integer`
- `riskScore : Float`
- `requiredSkill : String`

Context

- `urgencyWeight : Float`
- `valueWeight : Float`
- `alignmentWeight : Float`

SprintPlan

- `planId : String`
- `totalValue : Float`
- `totalRisk : Float`
- `selectedStories : List<storyId>`

Explanation

- `reason : String`
- `confidenceScore : Float`

5. ComponentDesign

For each component we provide: Inputs, Outputs, Responsibilities, Core processing steps, Exceptions.

5.1 ContextExtractor

Inputs: historical sprint dataset (CSV/JSON), optional Jira export

Outputs: `Context` object, cleaned dataset

Responsibilities & steps (pseudocode):

```
function extractContext(dataset):
    validate(dataset)
    normalize_fields()
    compute team_velocity
    compute average_completion_rate
    detect dependencies and tags
    produce Context(urgency, value, alignment, skill_distribution)
    return Context
```

Exceptions: malformed file, missing required fields → raise `DataValidationError`.

5.2 WeightLearningModel

Inputs: Context, historical outcomes (completion, delays)

Outputs: prioritization weights (urgencyWeight, valueWeight, alignmentWeight)

Processing:

```
function trainModel(context, history):
    extract features per sprint
    fit regression/classifier or compute heuristics
    output normalized weights
```

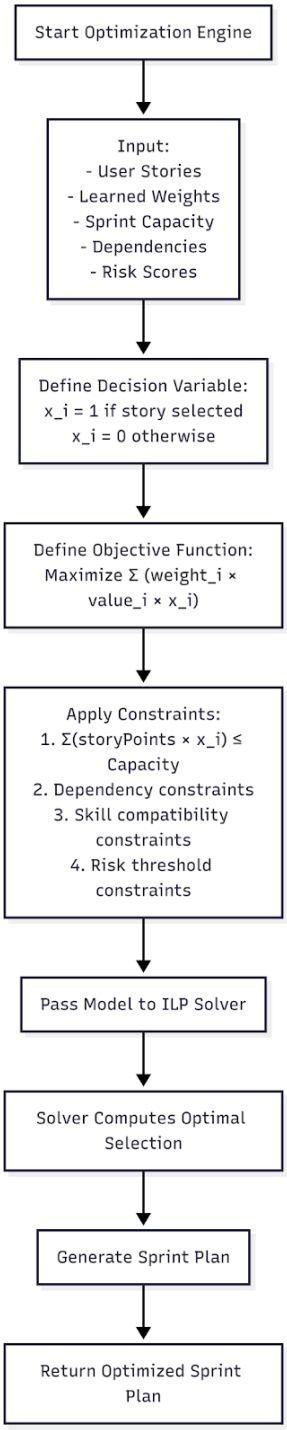
Notes: model can be a simple ML regressor or rule-based estimator depending on dataset size.

5.3 OptimizationEngine

Inputs: candidate UserStories, learned weights, capacity, dependencies, risk/skill scores
Outputs: SprintPlan (selected story list, objective value)
Nominal algorithm (pseudocode):

```
function optimizeSprintPlan(stories, weights, capacity):
    define objective = sum(weights * story_value)
    subject to:
        sum(story.storyPoints) <= capacity
        for each dependency: if child selected then parent selected
        optional: skill_assignment constraints
    solve using ILP solver or heuristic (e.g., knapsack + dependencies)
    return SprintPlan
```

Exceptions: infeasible due to constraints → return best-effort plan + warning.



Optimization Engine Flow illustrating ILP formulation, objective definition, constraint application, and sprint plan generation.

5.4 ExplainabilityEngine

Inputs: SprintPlan, story features, weights

Outputs: Explanation records for each selected story

Processing:

```
function generateExplanation(plan, weights):
    for story in plan.selectedStories:
        compute contribution = weighted_factors(story, weights)
        build textual justification
    return list of Explanations
```

Format: structured JSON: `{ storyId, reason, contributionBreakdown, confidenceScore }`

6. HumanInterfaceDesign

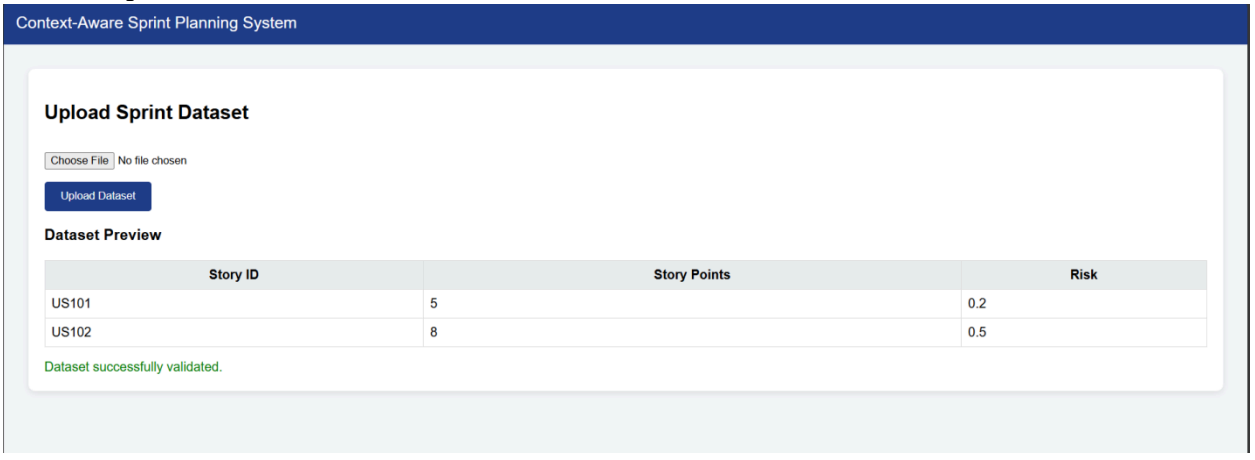
6.1 Overview and Screen Images

Primary screens:

- Dataset Upload
- Sprint Configuration (goals, capacity, constraints)
- Generated Sprint Plan (ranked list + explanations)
- Plan Review & Approve (manual edit + export)

Diagram placeholders (UI mockups):

Dataset Upload Screen



Generated Plan + Explainability

6.3 Screen Objects and Actions

Object	Action
Upload Button	Imports dataset
Generate Plan	Triggers optimization
Approve Plan	Exports to Jira
Modify Plan	Opens editor and re-runs optimizer

.

7. RequirementsMatrix

Map SRS to SDS components (traceability):

SRS Requirement	SDS Component
F1 Import Data	ContextExtractor
F2 Context Analysis	ContextExtractor
F3 Learn Weights	WeightLearningModel
F4 Optimization	OptimizationEngine
F5 Risk Awareness	OptimizationEngine
F6 Explainability	ExplainabilityEngine
F7 Modification	UI + OptimizationEngine
F8 Reporting	UI / Reporting Module

8. Appendices

Appendix A – Data Dictionary (`tbl_data_dictionary.png`)
Appendix B – Algorithm Details (`fig_algo_optimization.png`, pseudocode full listing)
Appendix C – UML Diagrams

- `fig_class_diagram.png`
- `fig_sequence_generate_plan.png`
- `fig_activity_generate_plan.png`
- `fig_usecase.png`

Appendix D – Project Planning (schedule & charts)

- `fig_gantt.png`
- `fig_pert.png`
- `fig_wbs.png`

Appendix E – Revision History (`tbl_revision_history.png`)